



香港科技大學
THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

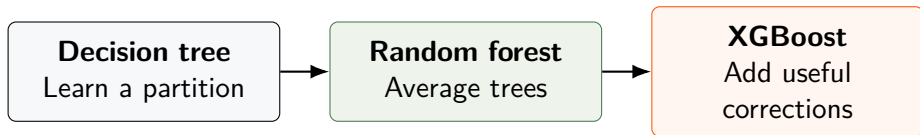
COMP 5212: Machine Learning

Decision Trees, Random Forests, and XGBoost

Instructor: Zihan Zhang

Supervised Learning

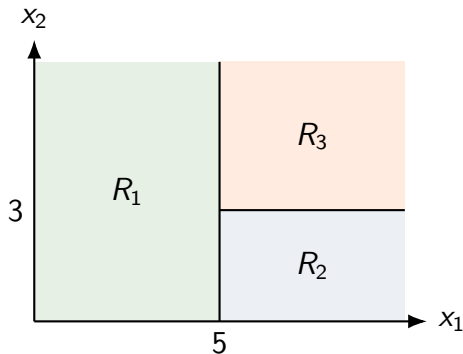
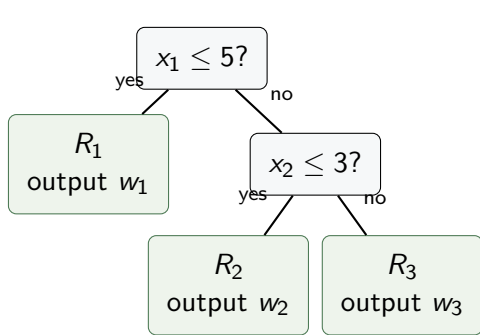
From One Tree to Two Ensembles



- Construct splits and compute leaf predictions.
- Explain feature randomization and out-of-bag evaluation.
- Derive XGBoost leaf values and split gain; examine recent applications.

80 minutes: introduction 3; trees 23; forests 17; boosting 33; discussion 4.

A Tree Partitions the Feature Space



$$f(x) = \sum_{v=1}^J w_v \mathbf{1}\{x \in R_v\}.$$

Each leaf defines a region with a constant prediction.

What Does a Leaf Predict?

Regression

For squared error, use the mean target in the leaf:

$$w_v = \frac{1}{|I_v|} \sum_{i \in I_v} y_i.$$

Targets: 2, 4, 6

Prediction: 4

Classification

Estimate class proportions in the leaf:

$$p_v(c) = \frac{\sum_{i \in I_v} \mathbf{1}\{y_i = c\}}{|I_v|}.$$

Labels: 0, 1, 1, 1

$p_v(1) = 0.75$;

predict class 1

I_v : training samples reaching leaf v ; J : number of leaves.

Regression: Choose the Split That Reduces SSE

Feature x	1	2	3	4
Target y	1	2	8	9

One leaf

Split at $x \leq 2.5$

$$\bar{y} = 5, \quad \text{SSE} = 50.$$

$$w_L = 1.5, \quad w_R = 8.5,$$

$$\text{SSE}_L + \text{SSE}_R = 1.$$

$$(j^*, a^*) = \arg \min_{j, a} \left[\sum_{i: x_{ij} \leq a} (y_i - \bar{y}_L)^2 + \sum_{i: x_{ij} > a} (y_i - \bar{y}_R)^2 \right].$$

Search features and thresholds; fit each candidate's leaf means.

Classification: Choose Purer Children

For class proportions $p_1, \dots, p_C$, the Gini impurity is

$$\text{Gini}(I) = 1 - \sum_{c=1}^C p_c^2.$$

Node	Class 0	Class 1	Gini
Parent	4	4	0.50
Left child	3	1	0.375
Right child	1	3	0.375

$$\Delta = \text{Gini}(I) - \frac{|I_L|}{|I|} \text{Gini}(I_L) - \frac{|I_R|}{|I|} \text{Gini}(I_R) = 0.125.$$

Select the candidate with the largest impurity decrease.

Grow the Tree One Split at a Time

1. Start with all training samples at the root.
2. At a node, evaluate candidate feature–threshold pairs.
3. Choose the best allowed split and send samples to its children.
4. Repeat until a stopping condition is reached; assign leaf outputs.

Local search

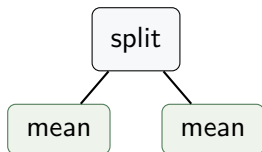
An early split changes every later decision in its subtree.

Computational shortcut

Sort a numerical feature and scan candidate cut points.

Greedy splitting does not guarantee the globally best tree.

Control How Much the Tree Can Fit



Few broad regions

- Limit maximum depth.
- Require enough samples in each leaf.
- Prune a larger tree using a complexity penalty.

Pruning objective: $R(T) + \alpha |\text{leaves}(T)|$.

$R(T)$: total training loss or impurity across leaves.

Choose complexity on validation data; deeper trees can have higher variance.

Learn Where Missing Values Should Go

For a candidate split $x \leq 5$, compare both missing-value directions.

x	1	2	8	9	missing	missing
y	0	0	1	1	1	1

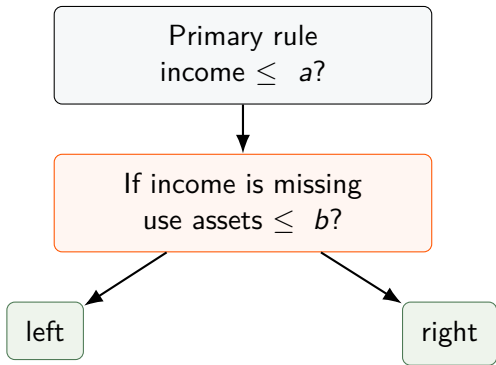
Missing direction	Left labels	Right labels	Weighted Gini
Left	0, 0, 1, 1	1, 1	1/3
Right	0, 0	1, 1, 1, 1	0

Learn (feature, threshold, missing direction) at each node.

This is a routing rule. Later splits can still separate missing samples.

Support is implementation-dependent.

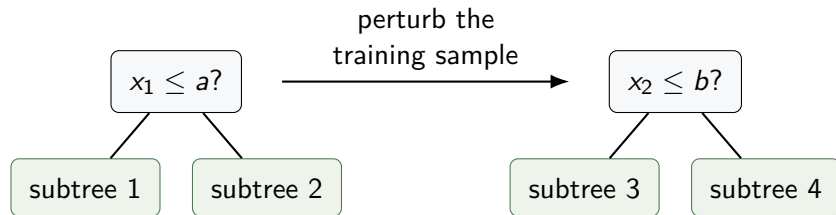
A Surrogate Split Uses Another Feature



- Find a rule that reproduces the primary split's left/right assignments.
- Samples missing the primary feature may take different branches.
- A learned default direction can also separate samples later, using other features.

CART-style surrogate splits; illustrative rules.

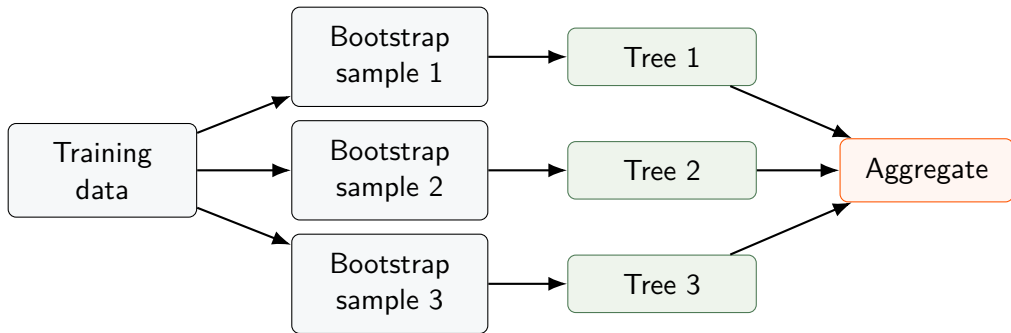
A Small Data Change Can Change the Tree



- Competing splits may have very similar training scores.
- A different early split changes the downstream partition.
- Averaging many diverse trees can stabilize predictions.

Schematic illustration of tree instability. Next: random forests.

Random Forest: Two Sources of Randomness



- Sample training rows with replacement for each tree.
- At each node, search a random subset of the features.

Regression: average predictions. Classification: vote or average class probabilities, depending on implementation.

Random Features Reduce Shared Errors

Suppose one feature dominates the split score in many bootstrap samples.

Search every feature

Many trees select similar early splits.

Their prediction errors can be strongly correlated.

Search a random subset

Some nodes must use alternative predictors.

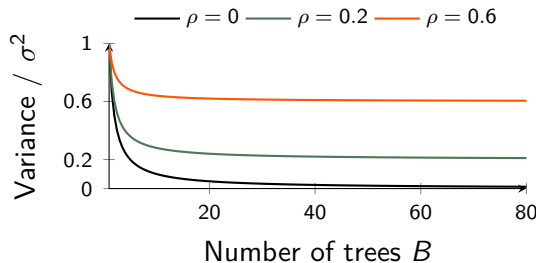
Trees can become more diverse.

Tradeoff: fewer candidate features can reduce correlation, but may also weaken individual trees.

Correlation Limits the Gain from Averaging

At a fixed input, suppose trees have variance σ^2 and pairwise correlation ρ .

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B T_b(x)\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$



More trees reduce the second term.

Lower correlation reduces the limiting variance.

An explanatory equal-variance/equal-correlation model; the plot is calculated from this formula.

OOB Is Defined Separately for Each Tree

	Tree 1	Tree 2	Tree 3	Tree 4
Sample A	in-bag	OOB	in-bag	OOB
Sample B	OOB	in-bag	in-bag	in-bag
Sample C	in-bag	in-bag	OOB	OOB

For sample A, aggregate predictions from trees 2 and 4 only.

$$\hat{f}_{\text{OOB}}(x_i) = \frac{1}{|\mathcal{B}_i|} \sum_{b \in \mathcal{B}_i} T_b(x_i), \quad \mathcal{B}_i = \{b : i \notin D_b\}, \quad |\mathcal{B}_i| > 0.$$

Compare each sample's OOB prediction with its observed target.

Formula shown for regression; classification can aggregate OOB class probabilities.

OOB Depends on Draws per Tree

Original data: n samples. Each tree draws m times with replacement.

$$q = P(i \text{ is OOB for one tree}) = \left(1 - \frac{1}{n}\right)^m \approx e^{-m/n}.$$

Draws per tree	OOB probability q	Expected OOB trees, $B = 100$
$m = n$	36.8%	36.8
$m = 2n$	13.5%	13.5
$m = 5n$	0.67%	0.67

Increasing B gives more OOB predictions per sample.

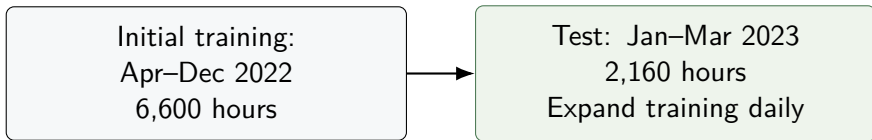
Increasing m/n makes exclusion from any one tree less likely.

Large- n approximations. Standard bootstrap usually takes $m = n$; $m > n$ rows are illustrative variants.

Random Forest for Port Energy Forecasting

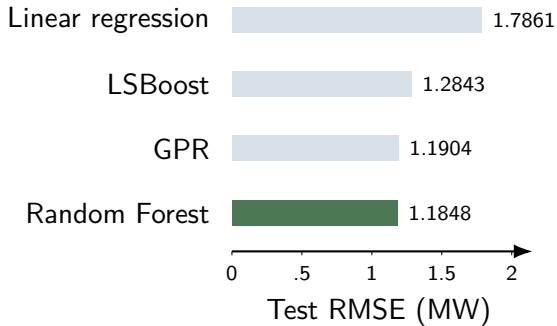
Valletta Cruise Port, Malta: a 2025 study

- Predict next-day hourly demand to support energy scheduling.
- Inputs: past demand, calendar, temperature, PV generation, ship activity.
- 8,760 hourly records; metered baseline plus estimated shore-power demand.



Retrospective evaluation.

Lower Forecast Error Than Linear Regression



33.7%

lower RMSE than linear regression.

GPR was nearly tied.

All shown models use expanding-window evaluation.

Some future covariates were observed retrospectively. No measured cost saving.

GPR: exponential kernel. Reduction calculated from reported RMSE.

Boosting Learns Successive Corrections

Random forest

Trees can be trained independently.

$$\hat{f}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x).$$

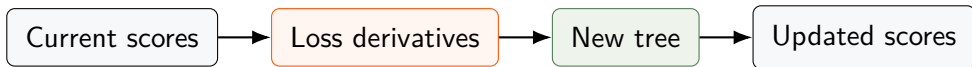
Average diverse predictions.

Gradient boosting

Each new tree depends on the current model.

$$F_t(x) = F_{t-1}(x) + \eta f_t(x).$$

Add a correction to the current score.



Connection to AdaBoost: sequential learning; the loss determines how later learners respond to errors.

Squared Loss Leads to Residual Fitting

Start with $F_0(x) = 5$. Fit a regression stump to $r_i = y_i - F_0(x_i)$.

x_i	1	2	3	4
y_i	1	2	8	9
Residual r_i	-4	-3	3	4
Stump $f_1(x_i)$	-3.5	-3.5	3.5	3.5
$F_1(x_i)$, with $\eta = 0.5$	3.25	3.25	6.75	6.75

$$F_1(x) = 5 + 0.5f_1(x).$$

The next tree fits what remains after this update.

Constructed example: unregularized squared loss. Later we restart it with XGBoost regularization.

A Tree Outputs a Score, Also for Classification

$$s(x) = b + \sum_{t=1}^T \eta f_t(x), \quad f_t(x) = w_{t,q_t(x)}.$$

Squared-error regression

$$\hat{y} = s,$$

$$\ell(y, s) = \frac{1}{2}(s - y)^2.$$

A leaf changes the predicted value.

Binary classification: $y \in \{0, 1\}$

$$p = \sigma(s) = \frac{1}{1 + e^{-s}},$$

$$\ell(y, s) = \log(1 + e^s) - ys.$$

A leaf changes the logit; sigmoid gives a probability.

$q_t(x)$ identifies a leaf; $w_{t,v}$ is that leaf's real-valued output.

One Round Optimizes One New Tree

Fix the existing ensemble, with current scores $s_i = F_{t-1}(x_i)$.

$$\mathcal{L}_t(f_t) = \sum_{i=1}^n \ell(y_i, s_i + f_t(x_i)) + \Omega(f_t).$$

$$\Omega(f_t) = \gamma J + \frac{\lambda}{2} \sum_{v=1}^J w_v^2.$$

- Learn the partition and its J leaf values.
- γ penalizes additional leaves; λ shrinks leaf values.
- First derive an unshrunk tree; apply learning rate η afterward.

L2-regularized scalar-tree formulation.

Approximate the Loss Around the Current Score

For each sample, expand in the proposed correction $f_t(x_i)$:

$$\ell(y_i, s_i + f_t(x_i)) \approx \ell(y_i, s_i) + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2.$$

$$g_i = \left. \frac{\partial \ell(y_i, s)}{\partial s} \right|_{s=s_i}, \quad h_i = \left. \frac{\partial^2 \ell(y_i, s)}{\partial s^2} \right|_{s=s_i}.$$

Drop terms that are constant with respect to the new tree:

$$\tilde{\mathcal{L}}_t = \sum_i \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t).$$

Exact for squared loss; a local approximation for logistic loss.

The Loss Supplies Two Numbers per Sample

Loss	Gradient g_i	Hessian h_i
$\frac{1}{2}(s_i - y_i)^2$	$s_i - y_i$	1
Binary cross-entropy	$p_i - y_i$	$p_i(1 - p_i)$

In classification, $p_i = \sigma(s_i)$.

Compute g_i, h_i from the existing ensemble once per round.

Keep them fixed while growing this round's tree.

Candidate leaves only need sums of these statistics.

Each Leaf Has an Optimal Constant Correction

For samples I_v reaching leaf v , define

$$G_v = \sum_{i \in I_v} g_i, \quad H_v = \sum_{i \in I_v} h_i.$$

Its contribution to the approximate objective is

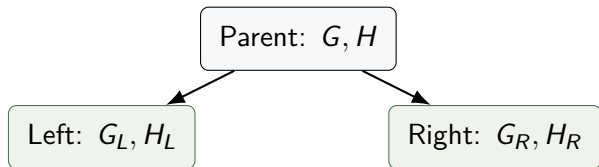
$$Q_v(w_v) = G_v w_v + \frac{1}{2}(H_v + \lambda)w_v^2 + \gamma.$$

$$\frac{\partial Q_v}{\partial w_v} = G_v + (H_v + \lambda)w_v = 0 \quad \implies \quad \boxed{w_v^* = -\frac{G_v}{H_v + \lambda}}.$$

For squared loss and $\lambda = 0$, this is the mean residual in the leaf.

A leaf value is a score correction contributed by this tree.

Split Gain Chooses the Tree Structure



$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right] - \gamma$$

- Compare the optimized parent leaf with two optimized child leaves.
- Search candidate features and thresholds; prefer the largest allowed gain.
- Recurse, with growth constraints and pruning controlling complexity.

Here $G = G_L + G_R$, $H = H_L + H_R$. The displayed gain already subtracts γ .

Worked Example: Start a Regularized Round

Restart from $s_i = 5$ for all samples. Use squared loss, $\lambda = 1$, $\gamma = 0$.

Feature x_i	Target y_i	Score s_i	$g_i = s_i - y_i$	h_i
1	1	5	4	1
2	2	5	3	1
3	8	5	-3	1
4	9	5	-4	1

$$G = 0, \quad H = 4.$$

Candidate thresholds: 1.5, 2.5, and 3.5.

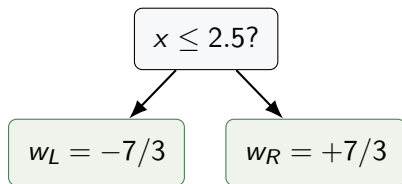
Constructed example. Set maximum depth to 1 to focus on a single split.

Compare the Candidates and Set Leaf Values

For $x \leq 2.5$: $G_L = 7$, $H_L = 2$, $G_R = -7$, $H_R = 2$.

$$\text{Gain} = \frac{1}{2} \left[\frac{49}{3} + \frac{49}{3} - \frac{0}{5} \right] = \frac{49}{3} \approx 16.33.$$

Threshold	Gain
1.5	6
2.5	16.33
3.5	6



The split groups samples that need similar corrections.

Shrink the New Tree, Then Update the Scores

$$F_t(x) = F_{t-1}(x) + \eta f_t(x).$$

x_i		1	2	3	4
With $\eta = 0.3$:	Old score	5	5	5	5
	New tree $f_t(x_i)$	$-7/3$	$-7/3$	$7/3$	$7/3$
	Applied correction	-0.7	-0.7	0.7	0.7
	New score	4.3	4.3	5.7	5.7
	Target	1	2	8	9

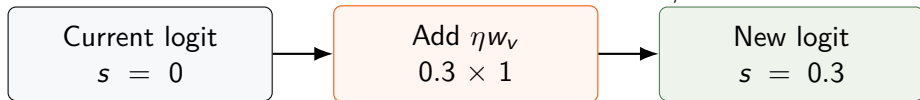
The next round recomputes gradients from these updated scores.

In Classification, Leaves Correct the Logit

A leaf contains labels $(1, 1, 1, 0)$; currently $s_i = 0$, so $p_i = 0.5$.

$$G_v = (-0.5) + (-0.5) + (-0.5) + 0.5 = -1, \quad H_v = 4(0.25) = 1.$$

With $\lambda = 0$, the leaf value is $w_v = -G_v/H_v = 1$.



$$p_{\text{new}} = \sigma(0.3) \approx 0.574.$$

Sum tree outputs first; apply sigmoid to the ensemble score.

One XGBoost Round, End to End

1. Compute current ensemble scores s_i and derivatives g_i, h_i .
2. Start a tree at the root; aggregate G, H for its samples.
3. Search feature–threshold candidates using split gain.
4. Continue growing subject to constraints; set final leaf values.
5. Update $s_i \leftarrow s_i + \eta w_{q_t(x_i)}$.

Recompute derivatives for the next tree.
Keep the existing trees fixed in this additive training scheme.

With subsampling, the tree is fitted using the selected rows/features. All training scores are then updated.

Regularization Acts at Several Levels

Control	What it changes
Maximum depth	Limits the complexity of interactions in one tree.
<code>min_child_weight</code>	Requires enough Hessian mass H in each child.
λ and γ	Shrink leaf values and penalize extra leaves.
Learning rate η	Shrinks each tree's contribution to the ensemble.
Row/feature subsampling	Adds randomness during tree construction.

Select boosting rounds on validation data.

Reserve the test set for final evaluation.

For squared loss with $h_i = 1$, Hessian mass equals sample count.

XGBoost for Early Sepsis Prediction

A 2023 study: predict sepsis within the next six hours.

- 4,603 trauma patients in MIMIC-IV; one hospital, 2008–2019.
- Clinical variables, measurement patterns, and time-series features.
- Compare XGBoost with L2-regularized logistic regression.

70% development
stratified 5-fold CV

15%
calibration

15%
held-out test

Random patient-level split: each patient's records stay together.

XGBoost vs. Logistic Regression

Six-hour prediction window; time-step-level evaluation.

Model	AUROC (95% CI)	AUPRC (95% CI)
L2 logistic regression	0.83 (0.81, 0.85)	0.18 (0.15, 0.21)
XGBoost	0.87 (0.85, 0.89)	0.27 (0.23, 0.31)

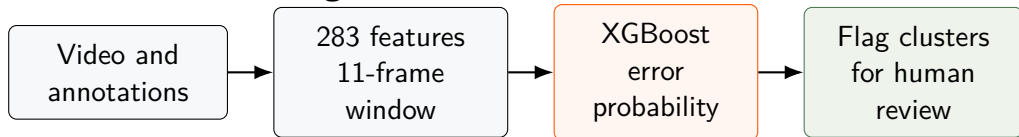
AUROC: area under the ROC curve.

AUPRC: area under the precision–recall curve. Higher is better.

Retrospective test results; clinical benefit and external generalization were not established.

XGBoost in Production: Uber, 2026

Detect bounding-box annotation errors before submission.



- Visual, motion, and geometric features summarize video context.
- Training anomalies simulate tracking jumps and identity swaps.
- Deployed across Uber's bounding-box annotation projects.

Deployment evidence; comparative metrics and savings not disclosed.

Three Models, Three Training Strategies

	Single tree	Random forest	XGBoost
Training	Greedy splits	Bootstrap + random features	Sequential loss-based corrections
Combine	One partition	Average or vote	Sum of score increments
Main benefit	Simple rules	Lower variance	Flexible loss minimization
Main controls	Depth, leaf size	Feature subset, leaf size, trees	Depth, regularization, learning rate, rounds

Choose using an evaluation that matches the intended deployment.

Check Your Understanding

1. A sample was used by 70 of 100 forest trees.

Which trees may contribute to its OOB prediction?

2. A binary XGBoost leaf has $w_v = -0.8$.

What does this do to the current score and positive-class probability?

Two samples reach this leaf with different current logits.

Are their probability changes equal?

Additional Details

Evaluate Models Under the Intended Use

- Split by time or group when observations are dependent.
- Fit preprocessing and tune hyperparameters using training/validation data.
- Compare a simple baseline, a tree, a forest, and boosted trees under the same protocol.
- Select a metric for the decision: error, ranking, calibration, or cost.
- Measure operational impact separately from offline predictive accuracy.

An OOB score is useful for suitable i.i.d. settings; it does not replace a time-aware or group-aware test.

A Newton Step as Weighted Least Squares

For $h_i > 0$, complete the square:

$$g_i f(x_i) + \frac{1}{2} h_i f(x_i)^2 = \frac{1}{2} h_i \left(f(x_i) + \frac{g_i}{h_i} \right)^2 - \frac{g_i^2}{2h_i}.$$

Pseudo-target

$$z_i = -\frac{g_i}{h_i}.$$

Sample weight

$$h_i.$$

Fit a tree to these weighted targets, with the tree regularizer added.

For squared loss, $h_i = 1$ and $z_i = y_i - s_i$: ordinary residuals.

Multiclass Scores and Softmax

Let $s_c(x)$ be the score for class $c \in \{1, \dots, C\}$.

$$p_c(x) = \frac{e^{s_c(x)}}{\sum_{k=1}^C e^{s_k(x)}}.$$

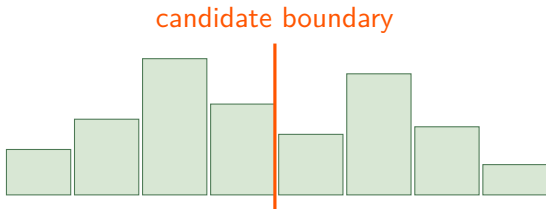
$$\ell(y, \mathbf{s}) = -\log p_y(x).$$

- Trees provide real-valued score increments for the classes.
- A common formulation fits one scalar-output tree per class per round.
- Predict $\hat{y} = \arg \max_c p_c(x)$.

This is the usual class-wise scalar-tree view. Multiclass curvature and vector-leaf variants require additional details.

Efficient Split Search Uses Histograms

1. Map each numerical feature into a set of bins.
2. Accumulate G and H in each bin.
3. Scan bin boundaries using prefix sums to evaluate split gain.



For missing values, compare adding their G , H to the left or right child.

Schematic histogram.

OOB Coverage Across the Forest

For independent bootstrap samples across B trees:

$$q = \left(1 - \frac{1}{n}\right)^m, \quad K_i = |\mathcal{B}_i| \sim \text{Binomial}(B, q).$$

$$\mathbb{E}[K_i] = Bq, \quad P(K_i = 0) = (1 - q)^B.$$

For large n , with $m = 5n$ and $B = 100$, each sample has about a 51% probability of having no OOB tree.

- OOB error evaluates subensembles with varying numbers of trees.
- Repeated tuning against the OOB score can overfit the evaluation.
- Time dependence and grouped observations need suitable validation splits.

An Operational Case: Virtua Health, 2023

XGBoost ran hourly within the electronic medical record workflow at five hospitals.

Reported operational metric	Value
XGBoost sensitivity	91%
XGBoost false-positive rate	6%
Existing commercial model's false-positive rate	30%

Three-week post-deployment evaluation: 1,896 patients, 98 sepsis cases.

A matched-sensitivity comparison was not reported.
Patient-outcome improvements were not established.

Optional discussion case.